

Digital Electronics (II SEM B.Tech. CSE)

Couse Code: NECE102



Dr. Swati Rajput

Assistant Professor

Department of Electronics Engineering

Indian Institute of Technology (Indian School of Mines) Dhanbad

Digital Electronics: Course Goals and Key Takeaways



Course Objective

To develop an understanding of digital electronics and its application which will be further useful to in the advanced courses of Electronics and Industrial Engineering.

Learning Outcomes

Upon successful completion of this course, students will:

- Gain information about the basic knowledge of digital electronics to design various types of digital circuits and systems.
- Learn how to analyse the performance and working of digital circuits.
- Develop their knowledge of digital electronics for industrial and robotic applications.

Digital Electronics: Key Topics at a Glance



Unit No.	Topics to be Covered	Lecture Hours	Learning Outcome
1.	Number System, Logic Gates, Boolean Algebra, NAND and NOR Implementation of Logic Functions, Determination of Boolean Functions from Digital Circuits.	08	Acquire an understanding of number system, basic logic gates and their applications to develop digital circuits and systems.
2.	Standard Representations of Logical Functions, SOP and POS form, Simplification of Logical Functions using K-map.	08	Acquire the knowledge to represent the logic functions in standard form and its minimization.
3.	Combinational Logic Circuit: Adder, Subtractor, Magnitude Comparator, Code Converters etc. MSI Circuits: Multiplexers, Demultiplexers, Decoders, Encoders etc.	08	Understand the design and applications of combinational logic circuits.
4.	Sequential Circuits: Latches and Flip-flops, Counters, Shift Registers.	10	Acquire the ability to design and perform the analysis of different sequential circuits.
5.	Analysis of Sequential circuits: State tables and state diagrams, Timing Circuits	04	Acquire an understanding on the analysis of sequential circuits.
6.	Introduction to logic families, semiconductor memories, ROM, PAL, PLA and Microprocessor	04	Develop knowledge about the various techniques for IC designing for digital circuits.

Textbook:	Reference Books:
1. Floyd T.L., Digital Fundamentals, 10/2, Pearson Education, 2011.	1. Digital Logic and Computer Design by M. Morris Mano, Prentice –Hall, 2015. 2. Fundamental of Digital Electronics by A. Anand Kumar, 4/e Prentice Hall, 2016.

Digital Electronics: Lecture Plan



Sl. No.	Topics	No of classes (tentative)
1.	Introduction to Number Systems and Conversion Techniques	8
2.	Continue.....	
3.	1's and 2's complement	
4.	Logic Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)	
5.	Boolean Algebra: Laws and Theorems	
6.	Boolean Function Implementation Using NAND/NOR Gates	
7.	Determination of Boolean Functions from Digital Circuits	
8.	Continue.....	
9.	SOP Standard Forms of Logical Functions	8
10.	POS Standard Forms of Logical Functions	
11.	Simplification of Logical Functions Using Boolean Algebra	
12.	Introduction to Karnaugh Maps (K-Maps)	
13.	K-Map Simplification (2-variable Functions)	
14.	K-Map Simplification (3-variable Functions)	
15.	K-Map Simplification (4-variable Functions)	
16.	Don't Care Conditions in K-Map Simplification	

Digital Electronics: Lecture Plan



17.	Introduction to Combinational Logic Circuits	8
18.	Design of Adders (Half Adder, Full Adder)	
19.	Design of Subtractors (Half Subtractor, Full Subtractor)	
20.	Magnitude Comparator Design	
21.	Code Converters (Binary to Gray, Gray to Binary, BCD to Excess-3)	
22.	Multiplexers: Design and Applications	
23.	Demultiplexers: Design and Applications	
24.	Decoders and Encoders	
25.	Introduction to Sequential Circuits	10
26.	Latches: SR Latch, D Latch	
27.	Flip-Flops: SR, JK Flip-Flops	
28.	Flip-Flops: D and T Flip-Flops	
29.	Asynchronous Counters: Design and Operation	
30.	Continue.....	
31.	Synchronous Counters: Design and Operation	
32.	Continue.....	
33.	Shift Registers: Types and Applications	
34.	Continue.....	

Digital Electronics: Weightage



Weightage of Different Components

1.	Quiz/ Assignment	10 Marks
2.	Mid Semester Examination	30 Marks
3.	Quiz/ Assignment	10 Marks
4.	End Semester Examination	50 Marks
Total Marks = 100		

Combinational Logic Design



- ❖ Logic circuits for digital systems may be combinational or sequential.
- ❖ **The output of a combinational circuit depends on its present inputs only.**
- ❖ Combinational circuits perform a specific information processing operation fully specified logically by a set of Boolean functions.
- ❖ **A combinational circuit consists of input variables, logic gates, and output variables.**
- ❖ The logic gates accept signals from the inputs and generate signals to the outputs. This process transforms binary information from the given input data to the required output data.
- ❖ Obviously, both input and output data are represented by signals, i.e. they exist in two possible values, one representing logic-1 and the other logic-0.
- ❖ **The n input binary variables come from an external source and the m output variables go to an external destination.**
- ❖ **For n input variables, there are 2^n possible combinations of binary input values.**
- ❖ **For each possible input combination, there is one and only one possible output combination.**
- ❖ A combinational circuit can be described by m Boolean functions one for each output variable.



Combinational Logic Design: Design Procedure



1. The problem is stated.
2. The number of available input variables and required output variables is determined.
3. The input and output variables are assigned letter symbols.
4. The truth table that defines the required relationship between inputs and outputs is derived.
5. The simplified Boolean function for each output is obtained.
6. The logic diagram is drawn.

Combinational Logic Design: Adders

- ❖ Digital computers perform a variety of information processing tasks.
- ❖ Among the basic tasks encountered are the various arithmetic operations.
- ❖ The most basic arithmetic operation is the addition of two binary digits. This simple addition consists of 4 possible operations, namely,

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad \text{and} \quad 1 + 1 = 10.$$

- ❖ The first three operations produce a sum whose length is one digit, but when both augend and addend bits are equal to 1, the binary sum consists of two digits.
- ❖ The higher significant bit of this result is called a carry. When the augend and addend numbers contain more significant digits, the carry obtained from the addition of two bits is added to the next higher-order pair of significant bits.

Combinational Logic Design: Adders



- ❖ A combinational circuit that performs the addition of two bits is called a *half-adder*.
- ❖ One that performs the addition of three bits (two significant bits and previous carry) is called a *full-adder*.
- ❖ The name of the former stems from the fact that two half-adders can be employed to implement a full-adder.

The Half Adder

- ❖ A half-adder is a combinational circuit with two binary inputs (augend and addend bits) and two binary outputs (sum and carry bits). It adds the two inputs (A and B) and produces the sum (S) and the carry (C) bits.
- ❖ It is an arithmetic circuit used to perform the arithmetic operation of addition of two single bit words.

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(a) Truth table



(b) Block diagram

The sum (S) bit and the carry (C) bit, according to the rules of binary addition, are given by: The sum (S) is the X-OR of A and B (It represents the LSB of the sum). Therefore,

$$S = A\bar{B} + \bar{A}B = A \oplus B$$

The carry (C) is the AND of A and B (It is 0 unless both the inputs are 1). Therefore,

$$C = AB$$

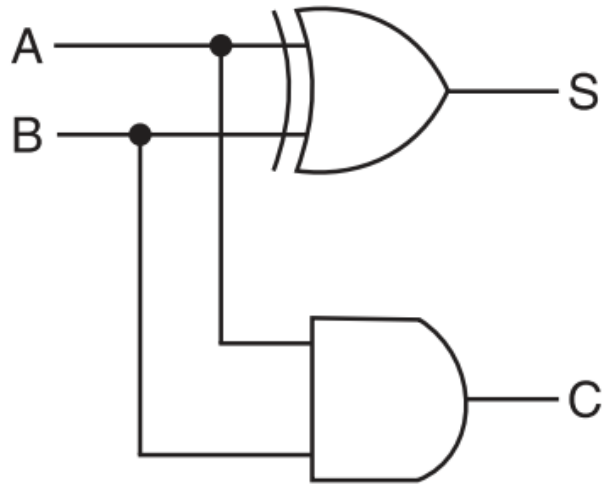
The Half Adder



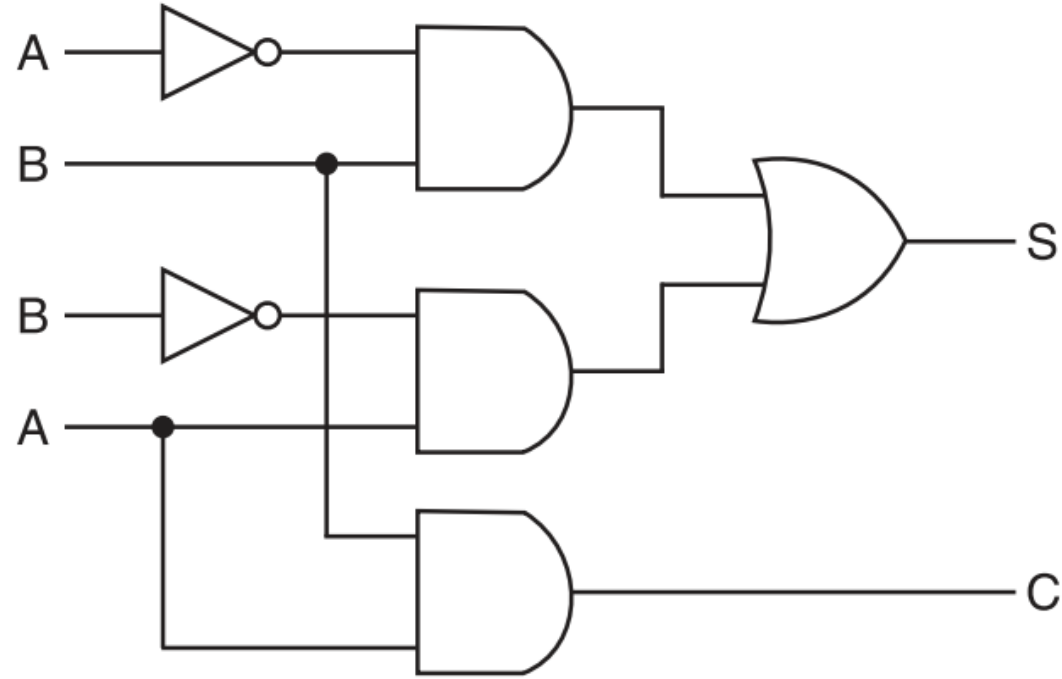
❖ A half-adder can, therefore, be realized by using one X-OR gate and one AND gate

$$S = A\bar{B} + \bar{A}B = A \oplus B$$

$$C = AB$$



(a)



(b)

The Half Adder using NAND Gate

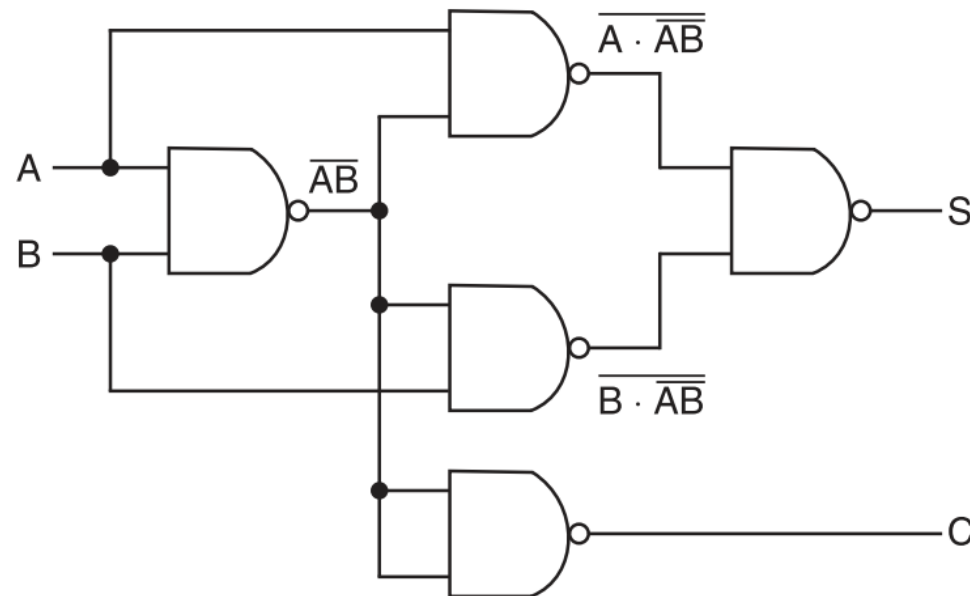


NAND logic

$$\begin{aligned} S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \\ &= A \cdot \overline{AB} + B \cdot \overline{AB} \\ &= \overline{\overline{A \cdot \overline{AB} \cdot B \cdot \overline{AB}}} \end{aligned}$$

$$C = AB = \overline{\overline{AB}}$$

$$C = AB = \overline{\overline{AB}}$$



The Half Adder using NOR Gate



NOR logic

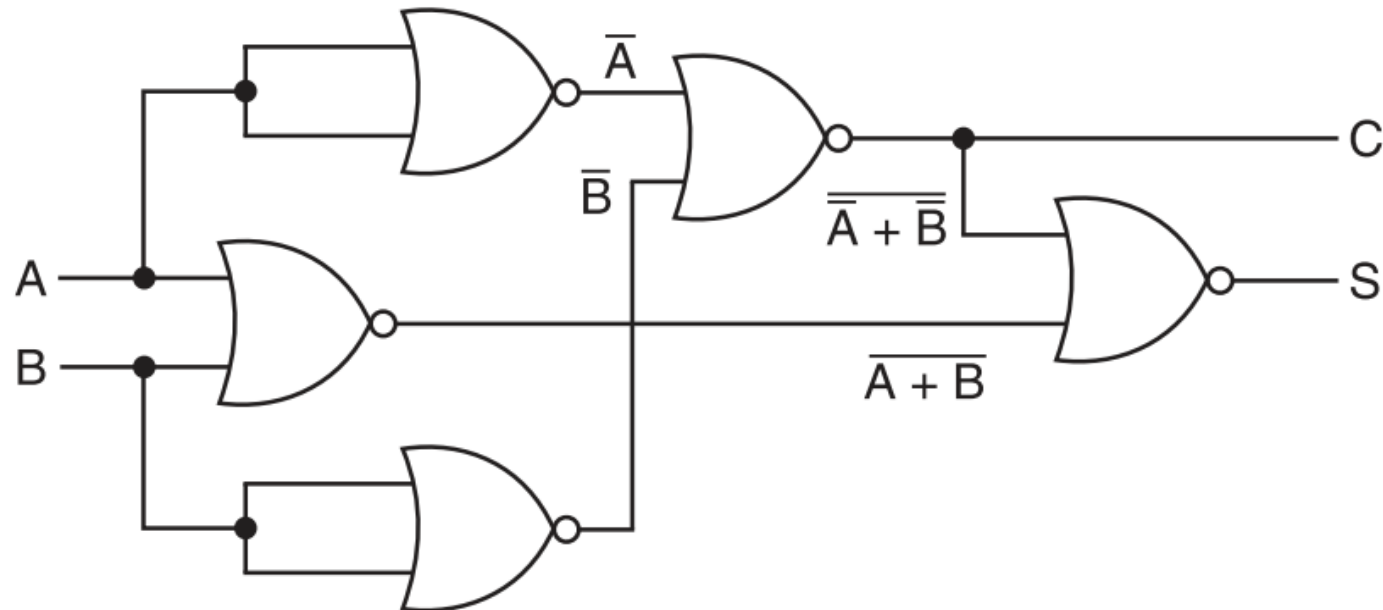
$$S = A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B}$$

$$= A(\bar{A} + B) + B(\bar{A} + \bar{B})$$

$$= (A + B)(\bar{A} + \bar{B})$$

$$= \overline{\overline{A + B} + \overline{\bar{A} + \bar{B}}}$$

$$C = AB = \overline{\overline{AB}} = \overline{\bar{A} + \bar{B}}$$



The Full Adder



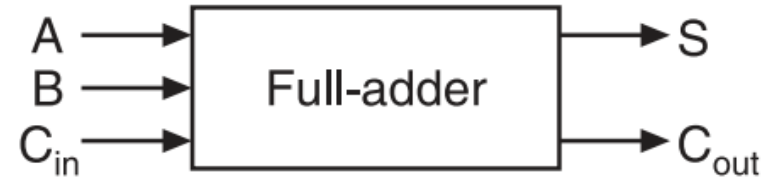
- ❖ A full-adder is a combinational circuit that adds two bits and a carry and outputs a sum bit and a carry bit.
- ❖ When we want to add two binary numbers, each having two or more bits, the LSBs can be added by using a half-adder.
- ❖ The carry resulted from the addition of the LSBs is carried over to the next significant column and added to the two bits in that column.
- ❖ So, in the second and higher columns, the two data bits of that column and the carry bit generated from the addition in the previous column need to be added.
- ❖ The full-adder adds the bits A and B and the carry from the previous column called the carry-in C_{in} and outputs the sum bit S and the carry bit called the carry-out C_{out} .
- ❖ The variable S gives the value of the least significant bit of the sum. The variable C_{out} gives the output carry.
- ❖ The eight rows under the input variables designate all possible combinations of 1s and 0s that these variables may have.
- ❖ The 1s and 0s for the output variables are determined from the arithmetic sum of the input bits.
- ❖ When all the bits are 0s, the output is 0.
- ❖ The S output is equal to 1 when only 1 input is equal to 1 or when all the inputs are equal to 1.
- ❖ The C_{out} has a carry of 1 if two or three inputs are equal to 1.

The Full Adder



Inputs			Sum	Carry
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a) Truth table



(b) Block diagram

From the truth table, a circuit that will produce the correct sum and carry bits in response to every possible combination of A, B, and C_{in} is described by

$$\begin{aligned} S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= (A\bar{B} + \bar{A}B)\bar{C}_{in} + (AB + \bar{A}\bar{B})C_{in} = (A \oplus B)\bar{C}_{in} + (\overline{A \oplus B})C_{in} = A \oplus B \oplus C_{in} \end{aligned}$$

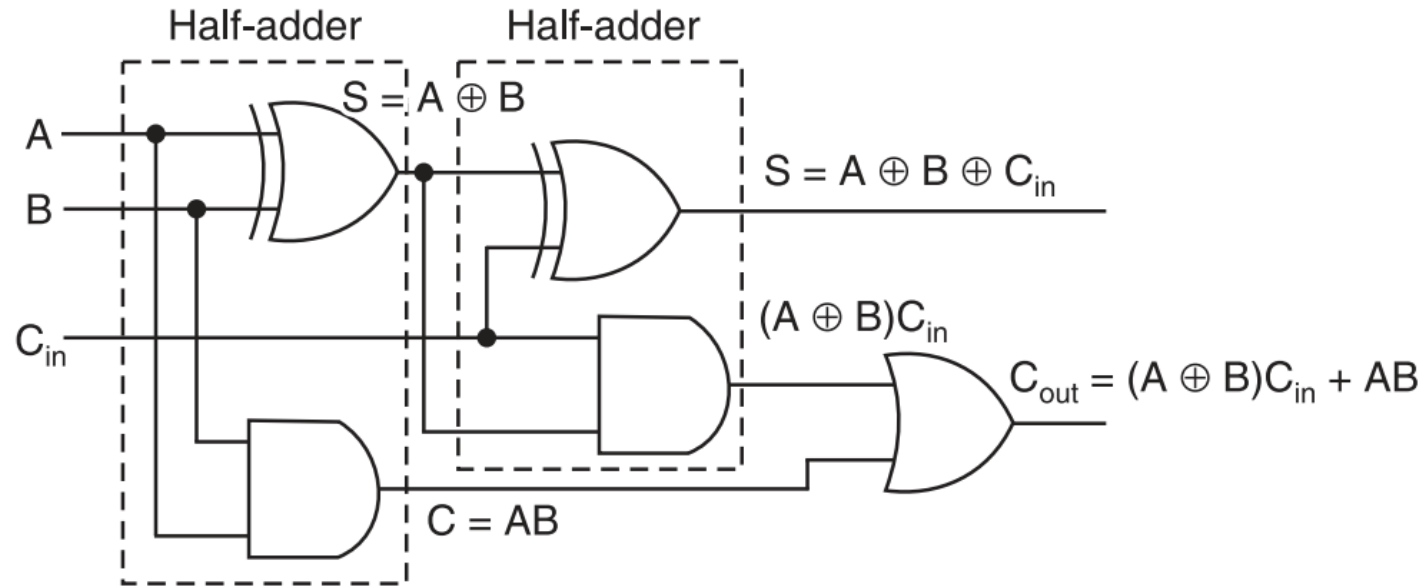
and

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} = AB + (A \oplus B)C_{in}$$

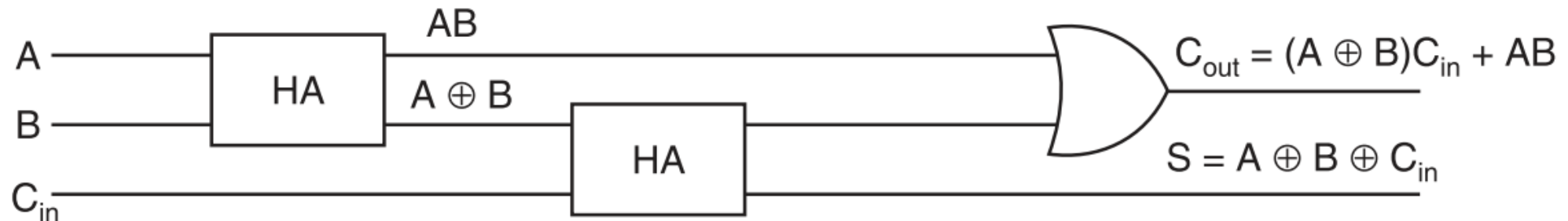
The Full Adder



Logic diagram of a full-adder using two half-adders



Block diagram of a full-adder using two half-adders



The Full Adder using NAND Gate

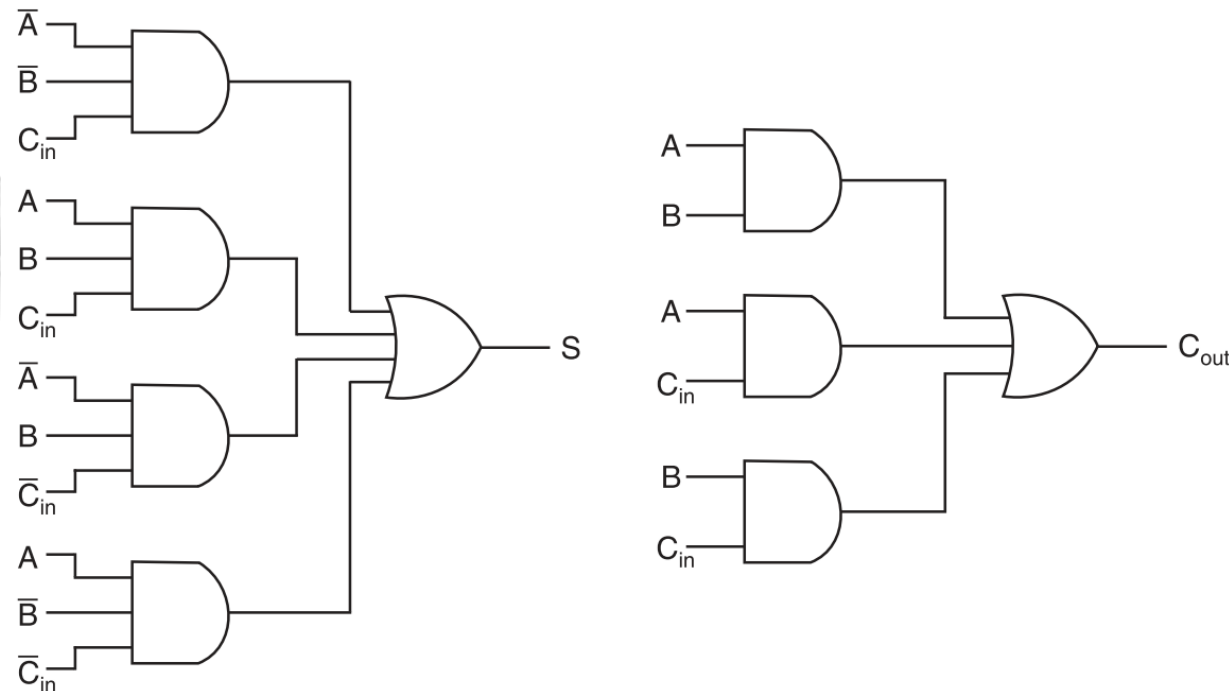


From the truth table, a circuit that will produce the correct sum and carry bits in response to every possible combination of A, B, and C_{in} is described by

$$\begin{aligned} S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= (\bar{A}\bar{B} + \bar{A}B)\bar{C}_{in} + (A\bar{B} + AB)C_{in} = (A \oplus B)\bar{C}_{in} + (\overline{A \oplus B})C_{in} = A \oplus B \oplus C_{in} \end{aligned}$$

and

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} = AB + (A \oplus B)C_{in}$$



The Full Adder using NAND Gate



NAND logic

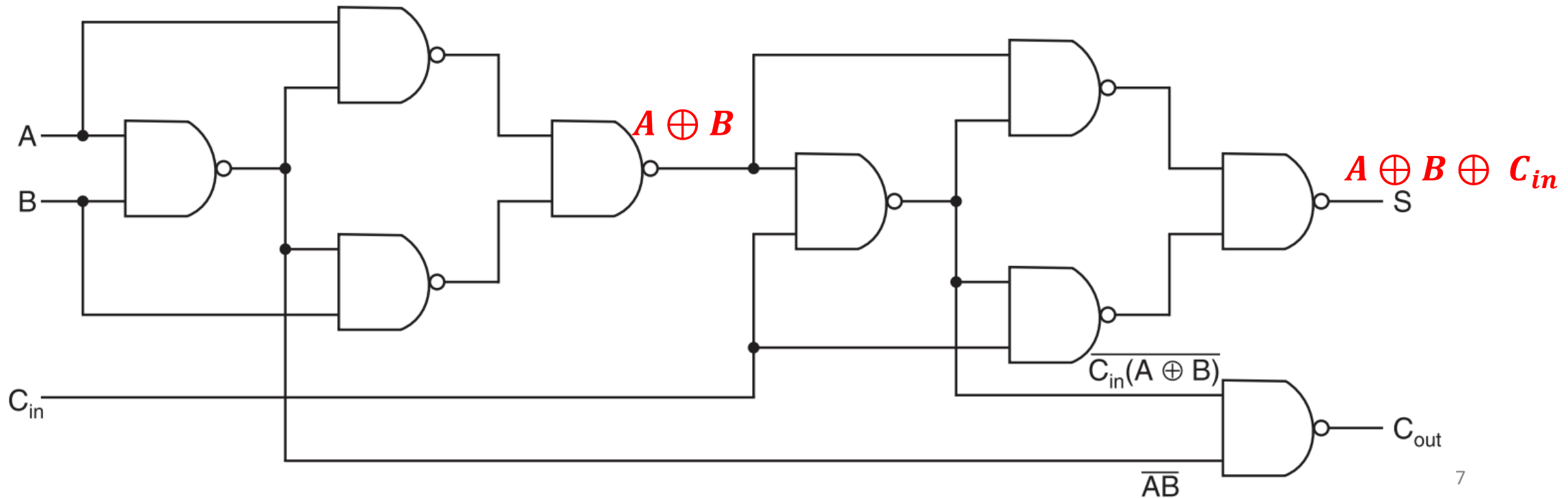
We know that

$$A \oplus B = \overline{\overline{A \cdot AB} \cdot \overline{B \cdot AB}}$$

Then

$$S = A \oplus B \oplus C_{in} = \overline{\overline{(A \oplus B) \cdot (A \oplus B)C_{in}} \cdot \overline{C_{in} \cdot (A \oplus B)C_{in}}}$$

$$C_{out} = C_{in}(A \oplus B) + AB = \overline{\overline{C_{in}(A \oplus B)} \cdot \overline{AB}}$$

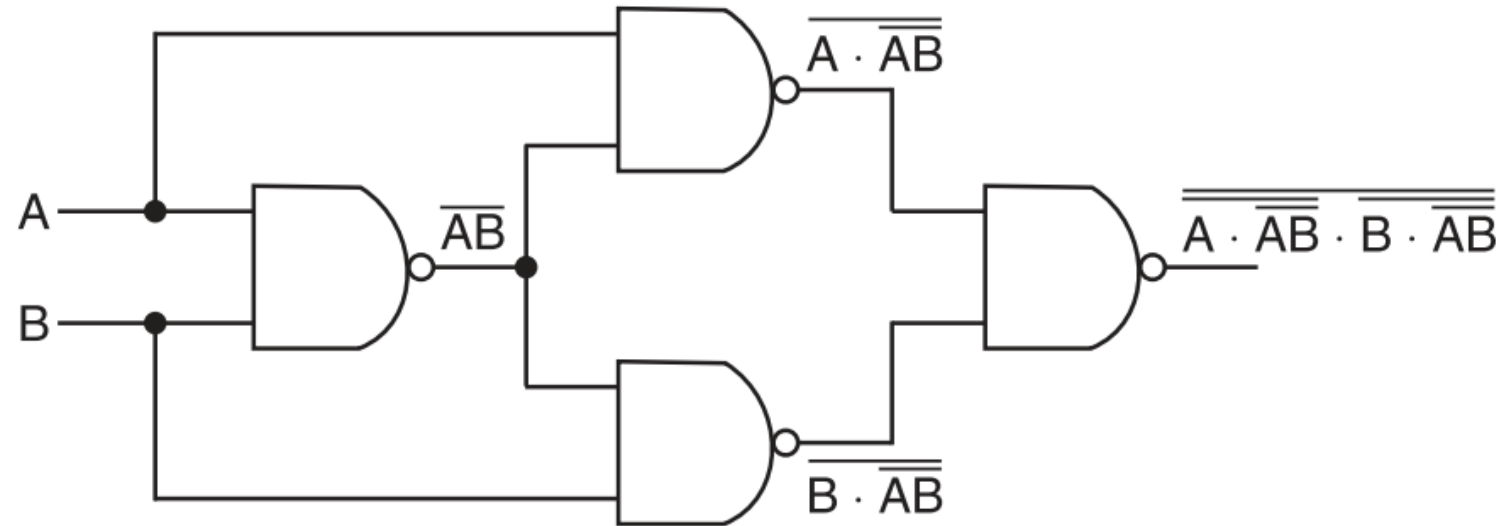


Boolean Algebra



EXAMPLE: Derive the Boolean expression for a two-input Ex-OR gate to realize with two input NAND gates without using complemented variables and draw the circuit.

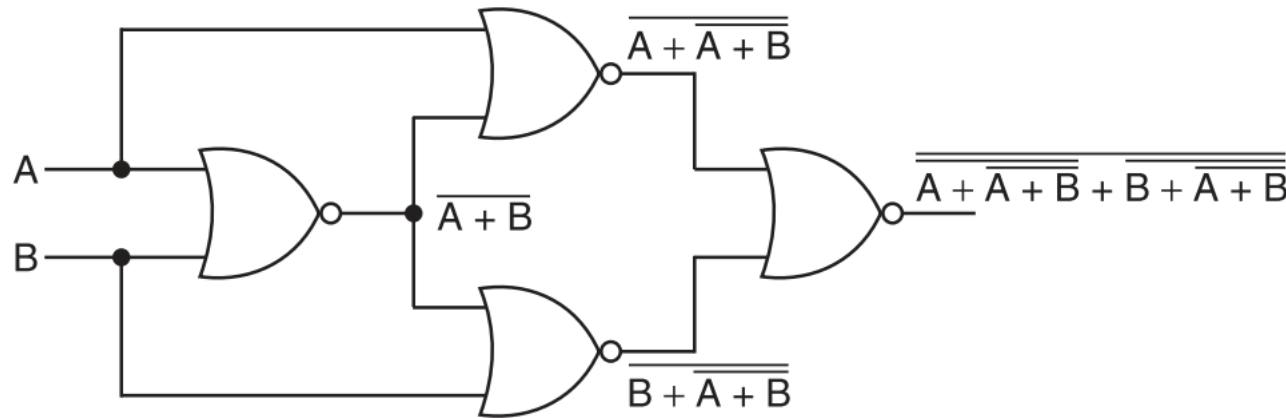
$$\begin{aligned} F &= A\bar{B} + \bar{A}B \\ &= A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \\ &= A \cdot \overline{AB} + B \cdot \overline{AB} \\ &= \overline{A \cdot AB \cdot B \cdot AB} \end{aligned}$$



Boolean Algebra



EXAMPLE: Derive the Boolean expression for a two-input Ex-NOR gate to realize with two input NOR gates, without using complemented variables and draw the circuit.



$$\begin{aligned} F &= \overline{A\bar{B}} + \overline{\bar{A}B} \\ &= \overline{A\bar{B} + B\bar{B} + \bar{A}B + A\bar{A}} \\ &= \overline{\bar{B}(A + B) + \bar{A}(A + B)} \\ &= \overline{\bar{B}(A + B) + \bar{A}(A + B)} \\ &= \overline{\bar{B} + A + B + A + A + B} \end{aligned}$$

The Full Adder using NOR Gate



NOR logic

We know that

$$A \oplus B = \overline{\overline{(A+B)} + \overline{\overline{A} + \overline{B}}}$$

Then

$$S = A \oplus B \oplus C_{in} = \overline{\overline{(A \oplus B) + C_{in}} + \overline{\overline{(A \oplus B) + C_{in}}}}$$

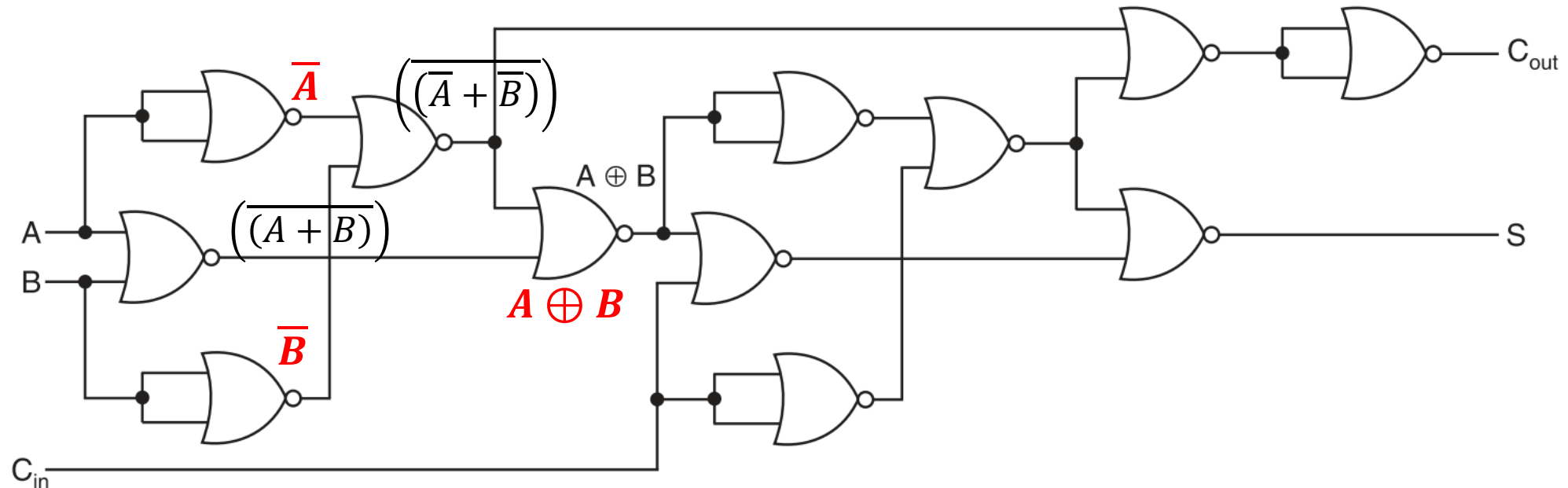
$$C_{out} = AB + C_{in}(A \oplus B) = \overline{\overline{A} + \overline{B} + \overline{C_{in}} + \overline{A \oplus B}}$$

$$A \oplus B = \overline{A}B + A\overline{B}$$

$$\begin{aligned} &= \overline{A}B + A\overline{A} + B\overline{B} + A\overline{B} \\ &= \overline{A}(A+B) + \overline{B}(A+B) \\ &= (A+B)(\overline{A} + \overline{B}) \end{aligned}$$

Take the double complementation law

$$\begin{aligned} A \oplus B &= \overline{\overline{(A+B)(\overline{A} + \overline{B})}} \\ &= \overline{\overline{(A+B)} + \overline{\overline{A} + \overline{B}}} \end{aligned}$$



The Half- Subtractor

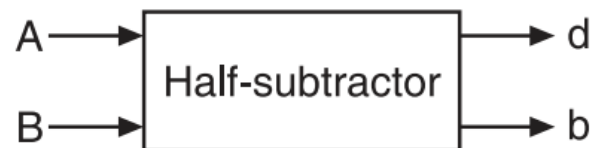
- ❖ A half-subtractor is a combinational circuit that subtracts one bit from the other and produces the difference.
- ❖ It also has an output to specify if a 1 has been borrowed.
- ❖ It is used to subtract the LSB of the subtrahend from the LSB of the minuend when one binary number is subtracted from the other.
- ❖ A half-subtractor shown is a combinational circuit with two inputs A and B and two outputs d and b.
- ❖ **d indicates the difference and b is the output signal generated that informs the next stage that a 1 has been borrowed.**
- ❖ We know that, when a bit B is subtracted from another bit A, a difference bit (d) and a borrow bit (b) result according to the rules given as follows
- ❖ The output borrow b is a 0 as long as $A \geq B$. It is a 1 for $A = 0$ and $B = 1$.

Inputs		Outputs	
A	B	d	b
0	0	0	0
1	0	1	0
1	1	0	0
0	1	1	1

(a) Truth table

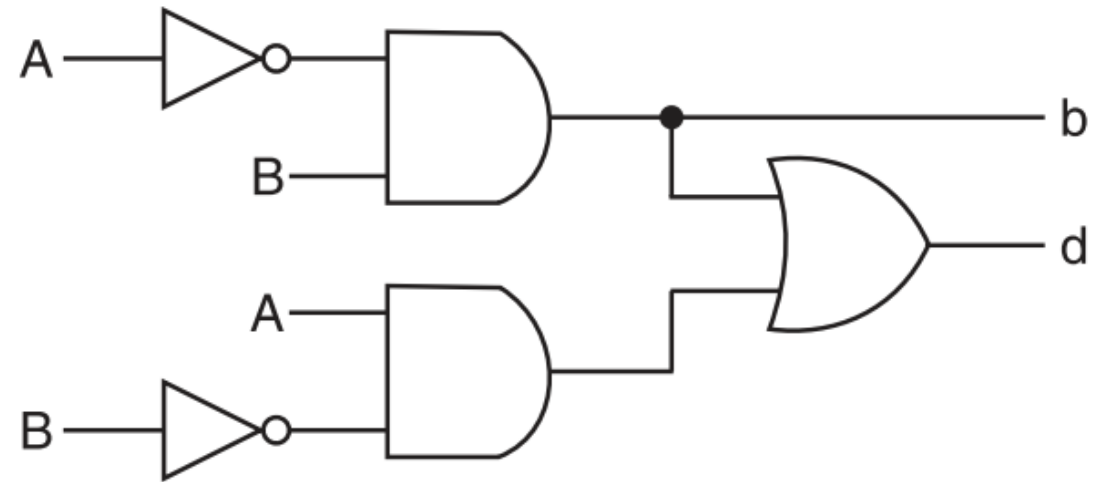
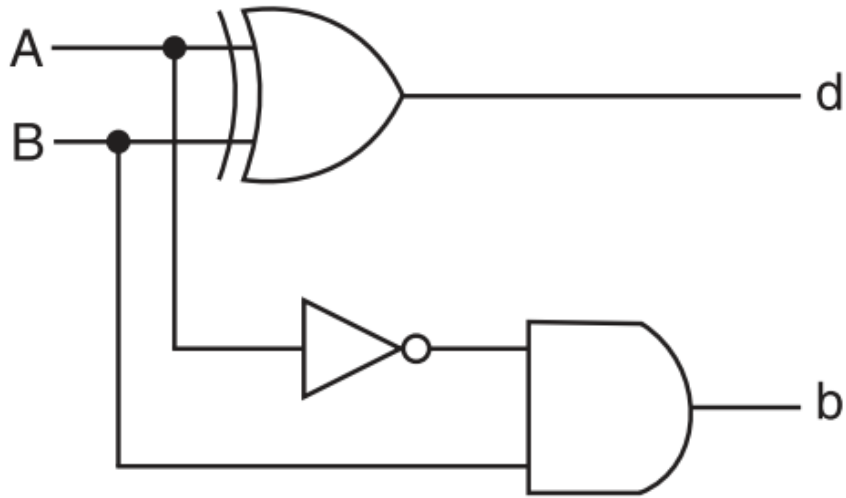
12-02-2026

$$d = A\bar{B} + \bar{A}B = A \oplus B \text{ and } b = \bar{A}B$$



(b) Block diagram

The Half- Subtractor



$$d = A\bar{B} + \bar{A}B = A \oplus B \text{ and } b = \bar{A}B$$

The Half- Subtractor using NAND Gate

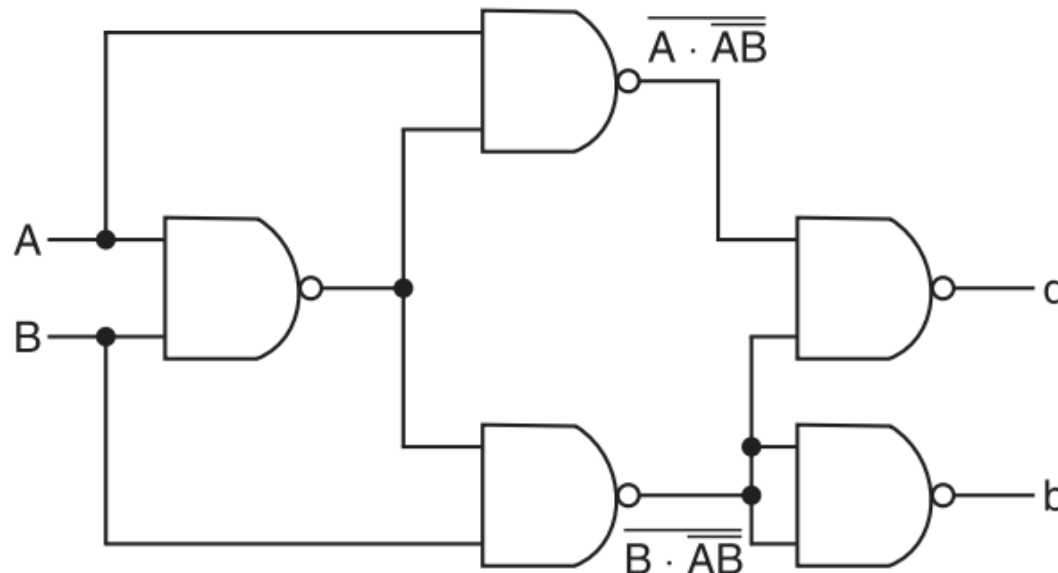


$$d = A\bar{B} + \bar{A}B = A \oplus B \text{ and } b = \bar{A}B$$

NAND logic

$$d = A \oplus B = \overline{\overline{A \cdot \bar{A}B} \cdot \overline{B \cdot \bar{A}B}}$$

$$b = \bar{A}B = B(\bar{A} + \bar{B}) = B(\overline{AB}) = \overline{B \cdot \bar{A}B}$$

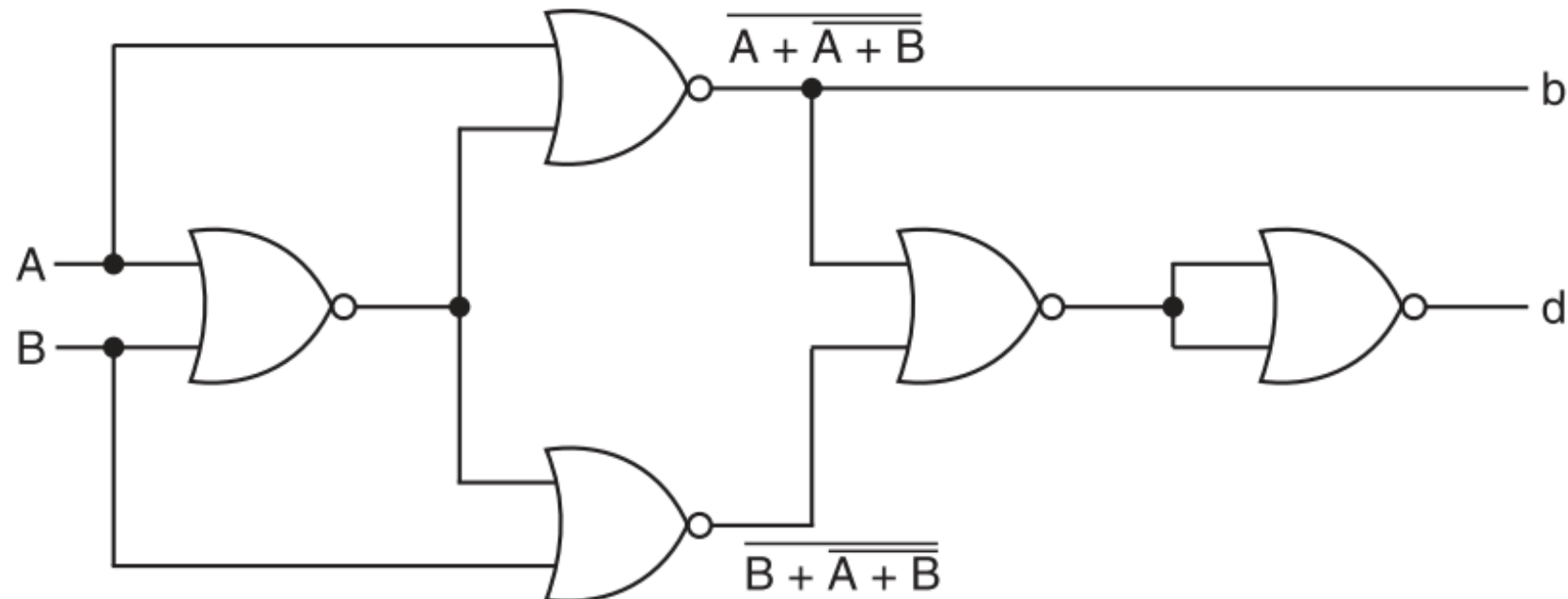


The Half- Subtractor using NOR Gate



NOR logic

$$\begin{aligned}d &= A \oplus B = A\bar{B} + \bar{A}B = A\bar{B} + B\bar{B} + \bar{A}B + A\bar{A} \\&= \bar{B}(A + B) + \bar{A}(A + B) = \overline{\overline{B} + \overline{A + B}} + \overline{\overline{A} + \overline{A + B}} \\d &= \bar{A}B = \bar{A}(A + B) = \overline{\overline{\bar{A}(A + B)}} = \overline{A + (\overline{A + B})}\end{aligned}$$

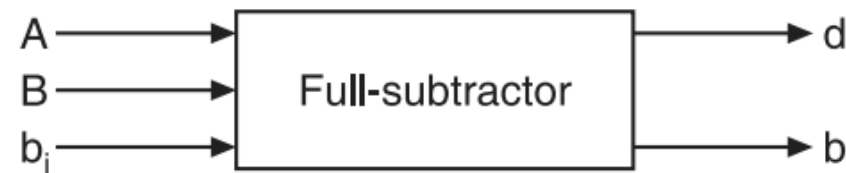


The Full - Subtractor

- It subtracts one bit (B) from another bit (A), when already there is a borrow b_i from this column for the subtraction in the preceding column, and outputs the difference bit (d) and the borrow bit (b) required from the next column.
- So, a full-subtractor is a combinational circuit with three inputs (A, B, b_i) and two outputs d and b.
- The two outputs present the difference and output borrow.
- The 1s and 0s for the output variables are determined from the subtraction of $A - B - b_i$.

Inputs			Difference	Borrow
A	B	b_i	d	b
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

(a) Truth table

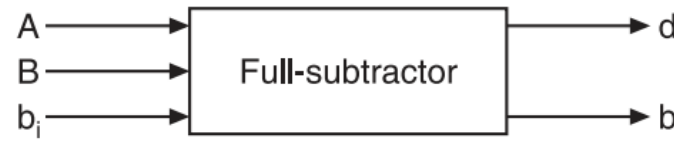


(b) Block diagram

The Full - Subtractor

Inputs			Difference	Borrow
A	B	b_i	d	b
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

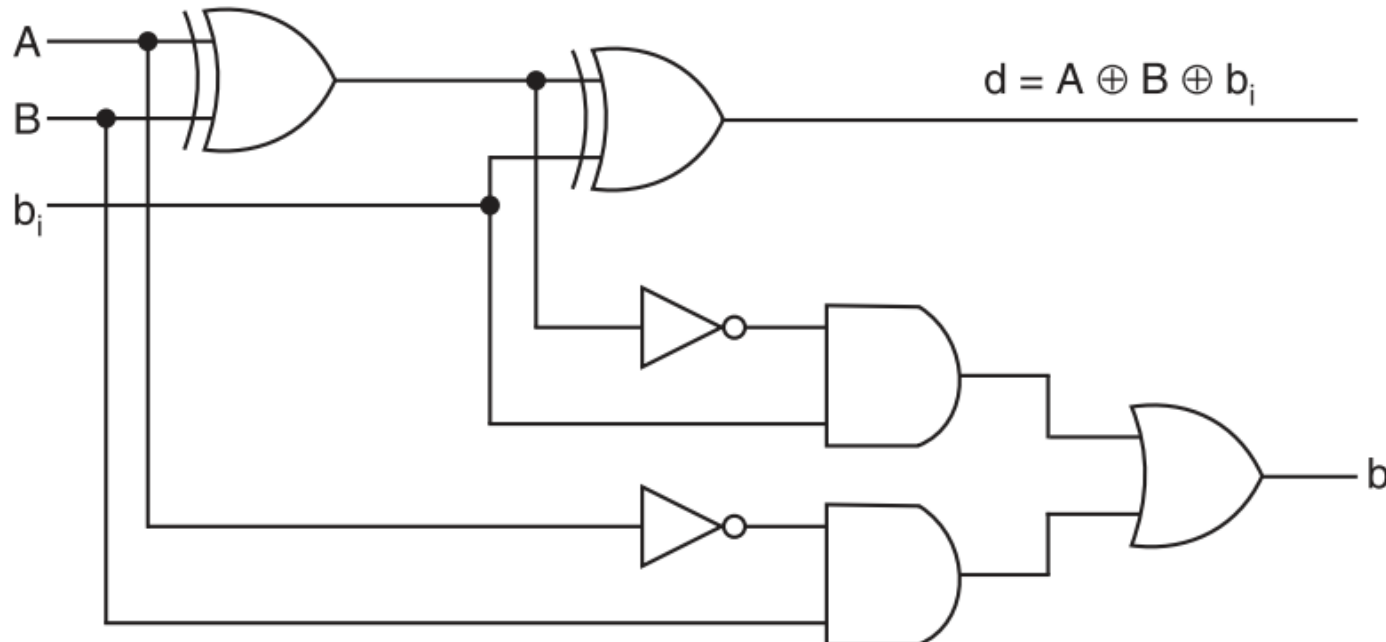
(a) Truth table



(b) Block diagram

$$\begin{aligned}
 d &= \bar{A}\bar{B}b_i + \bar{A}B\bar{b}_i + A\bar{B}\bar{b}_i + ABb_i \\
 &= b_i(AB + \bar{A}\bar{B}) + \bar{b}_i(A\bar{B} + \bar{A}B) \\
 &= b_i(\overline{A \oplus B}) + \bar{b}_i(A \oplus B) = A \oplus B \oplus b_i
 \end{aligned}$$

$$\begin{aligned}
 b &= \bar{A}\bar{B}b_i + \bar{A}B\bar{b}_i + \bar{A}Bb_i + ABb_i = \bar{A}B(b_i + \bar{b}_i) + (AB + \bar{A}\bar{B})b_i \\
 &= \bar{A}B + (A \oplus B)b_i
 \end{aligned}$$



The Full – Subtractor using NAND Gate

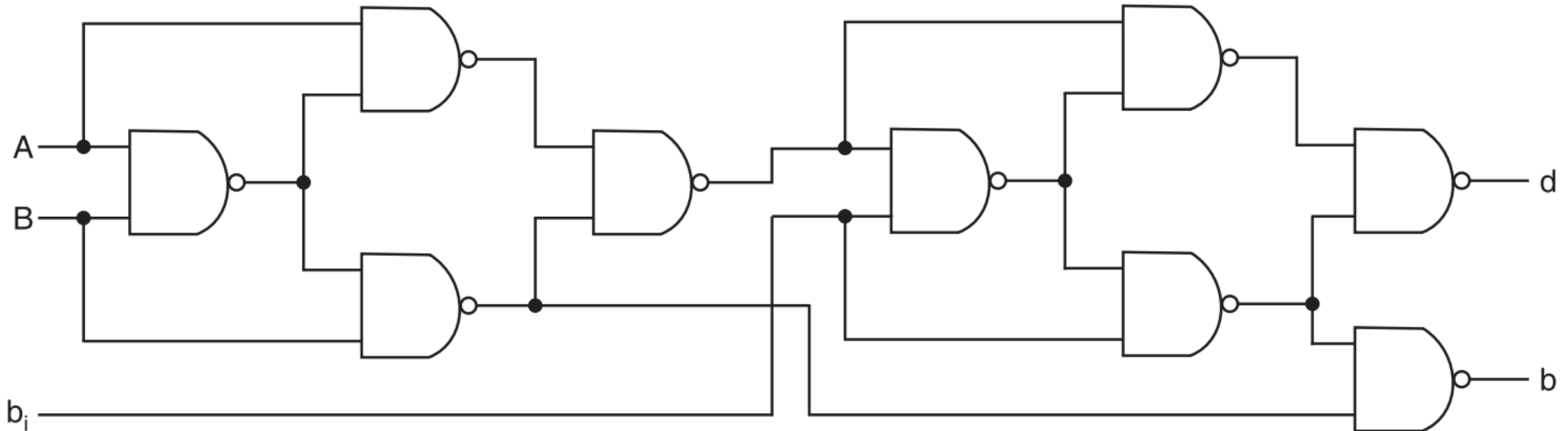


NAND logic

$$d = A \oplus B \oplus b_i = \overline{\overline{(A \oplus B) \oplus b_i}} = \overline{\overline{(A \oplus B)(A \oplus B)b_i} \cdot \overline{b_i(A \oplus B)b_i}}$$

$$\begin{aligned} b &= \overline{AB} + b_i(\overline{A \oplus B}) = \overline{AB} + b_i(\overline{A \oplus B}) \\ &= \overline{\overline{AB} \cdot b_i(\overline{A \oplus B})} = \overline{B(\overline{A} + \overline{B}) \cdot b_i(\overline{b_i} + (\overline{A \oplus B}))} \\ &= \overline{B \cdot AB \cdot b_i[b_i \cdot (\overline{A \oplus B})]} \end{aligned}$$

$$\begin{aligned} F &= A\overline{B} + \overline{A}B \\ &= A\overline{B} + A\overline{A} + \overline{A}B + B\overline{B} \\ &= A(\overline{A} + \overline{B}) + B(\overline{A} + \overline{B}) \\ &= A \cdot \overline{AB} + B \cdot \overline{AB} \\ &= \overline{A \cdot AB \cdot B \cdot AB} \end{aligned}$$

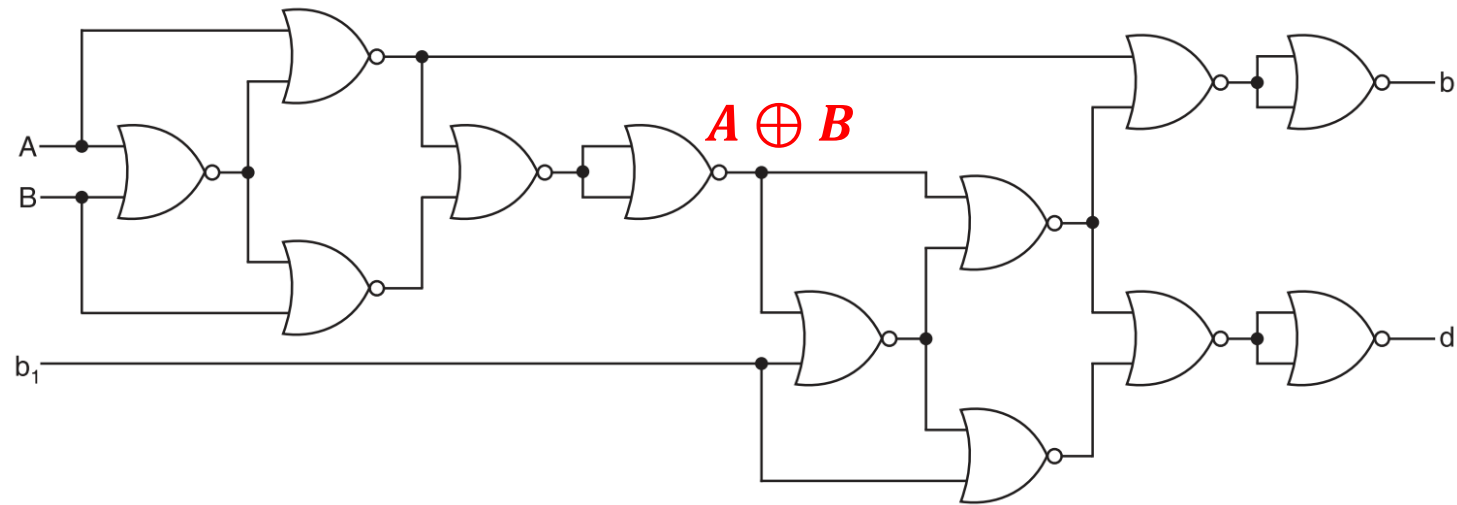


The Full – Subtractor using NOR Gate



NOR logic

$$\begin{aligned}
 d &= A \oplus B \oplus b_i = \overline{\overline{(A \oplus B) \oplus b_i}} \\
 &= \overline{(A \oplus B)b_i + (A \oplus B)\overline{b_i}} \\
 &= \overline{[(A \oplus B) + (A \oplus B)\overline{b_i}][b_i + (A \oplus B)\overline{b_i}]} \\
 &= \overline{(A \oplus B) + (A \oplus B) + b_i + b_i + (A \oplus B) + b_i} \\
 &= \overline{(A \oplus B) + (A \oplus B) + b_i + b_i + (A \oplus B) + b_i} \\
 b &= \overline{A}B + b_i(\overline{A \oplus B}) \\
 &= \overline{A}(A + B) + (\overline{A \oplus B})[(A \oplus B) + b_i] \\
 &= \overline{A} + (\overline{A + B}) + (A \oplus B) + (\overline{A \oplus B}) + b_i
 \end{aligned}$$

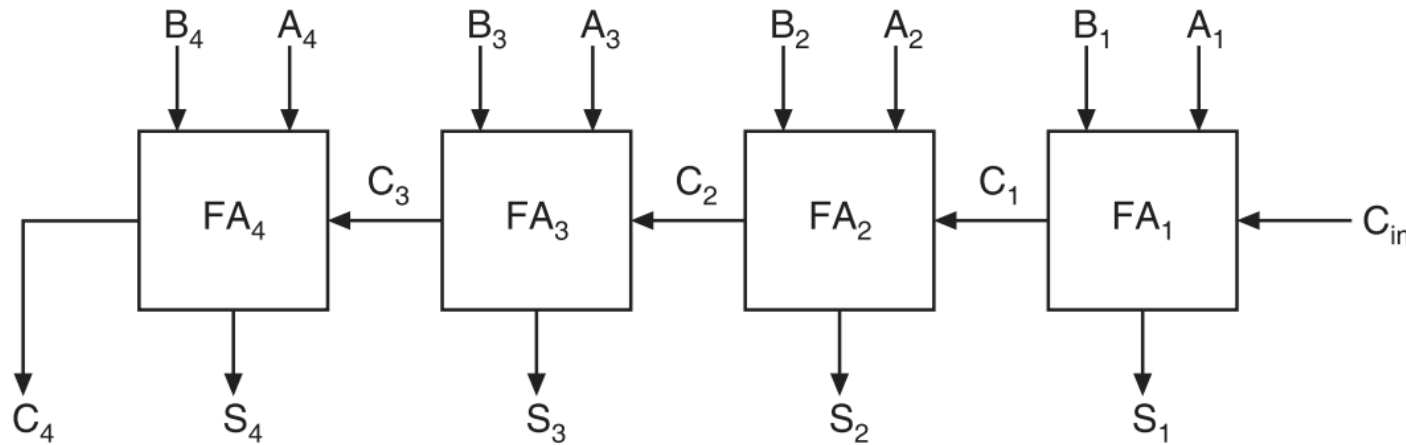


$$d = \overline{A}B = \overline{A}(A + B) = \overline{\overline{A}(A + B)} = \overline{A + (\overline{A + B})}$$

Binary Parallel Adder



- ❖ A binary parallel adder is a digital circuit that adds two binary numbers in parallel form and produces the arithmetic sum of those numbers in parallel form.
- ❖ It consists of full adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full adder in the chain.
- ❖ The interconnection of four full-adder (FA) circuits to provide a 4-bit parallel adder.
- ❖ The augend bits of A and addend bits of B are designated by subscript numbers from right to left, with subscript 1 denoting the lower-order bit.
- ❖ The carries are connected in a chain through the full-adders.
- ❖ The input carry to the adder is C_{in} and the output carry is C_4 . The S outputs generate the required sum bits.



Logic diagram of a 4-bit binary parallel adder.

An n -bit parallel adder requires n -full adders.